
Graph-on-Trees Generative Model

Abstract

We study an important, yet rarely studied, family of hierarchical graph data structures, *Graph-on-Trees* (ToG) and *Graph-of-Leaves* (ToL), as well as their temporal extensions, that represent relational data whose nodes are internal nodes and leaves, respectively, of trees. These structures capture patterns across accounting, banking, enterprise systems, amongst many others, where the actual relations occur at fine-grained leaves, while observations are frequently obtained at higher levels. Building on this formulation, we propose *ToG Diffusion* (ToG-D), a generative framework that stochastically coarsens leaf graphs into mixed-depth tree graphs, and trains a denoiser to reconstruct fine-grained relationships, using reconstruction and aggregation-consistency objectives.

1 Introduction

Many real-world relational datasets are not “flat” graphs: their nodes naturally live in a *hierarchy*. Accounts in a chart of accounts roll up into reporting categories; banking entities roll up through legal-entity and consolidation trees; products sit in taxonomies; organizations decompose into divisions, departments, and teams. In these settings, relations are often generated at the finest granularity (e.g., postings to leaf accounts, transactions between leaf entities), yet the data we observe, store, query, or report is frequently *aggregated* (e.g., monthly rollups, reporting-line summaries, multi-level dashboards). This creates a persistent mismatch between the granularity at which relations occur and the granularity at which they are available for modeling.

We study an important, yet rarely formalized, family of hierarchical graph data structures: *Graph-on-Trees* (ToG) and *Graph-of-Leaves* (ToL), along with their temporal extensions. A ToG is a graph whose nodes are allowed to be *any* nodes on a supporting tree (including internal nodes), whereas a ToL restricts nodes to the *leaves* of the tree. Temporal ToG/ToL represent time-indexed sequences (or event streams) of such graphs, enabling modeling of evolving hierarchical relations. These structures capture recurring patterns across accounting, banking, and enterprise systems, amongst many other domains: the underlying activity happens at fine-grained leaves, while observations and downstream tasks often involve mixed depths due to reporting requirements, privacy constraints, system design, or practical limits on data collection.

The hierarchical setting introduces modeling challenges that standard graph formalisms do not address well. First, aggregation is not merely missing data; it is a *structured* coarsening that merges multiple leaf nodes into shared ancestors and collapses many leaf-level edges into fewer coarse edges. Second, this coarsening is typically many-to-one, so inversion is *non-identifiable* without additional inductive bias. Third, the temporal nature of many applications (e.g., journal entries arriving as an event stream) implies that naïvely treating each time step as a graph can yield extremely sparse snapshots (sometimes a single edge), motivating windowed or event-based temporal formulations. A principled approach should exploit the tree structure, remain consistent with aggregation physics, and learn to recover fine-grained relational structure where possible.

Building on this formulation, we propose *ToG Diffusion* (ToG-D), a hierarchy-aware generative framework for modeling ToL/ToG datasets. ToG-D defines a forward corruption process that *stochastically coarsens* a leaf graph (ToL) into a mixed-depth tree graph (ToG) by lifting leaf nodes to ancestors

and aggregating edges accordingly, optionally injecting mild edge noise. The reverse model is trained as a denoiser that reconstructs fine-grained leaf-level relationships from the corrupted mixed-depth observation. Crucially, ToG-D is trained with a combination of (i) a leaf-level reconstruction objective and (ii) an *aggregation-consistency* objective that enforces that the reconstructed leaf graph, when re-aggregated through the same coarsening map, matches the observed coarse ToG. This consistency constraint anchors the learning problem to the observed data and mitigates degenerate solutions that ignore the hierarchical observation.

We further extend ToG-D by incorporating a pretrained graph foundation model, *GraphPFN*, as the backbone encoder. GraphPFN provides transferable structural priors and strong node representations, which we adapt to the hierarchical setting by combining tree edges with corrupted relational edges and decoding leaf-level directed edges. This yields a practical “pretrain + hierarchy-aware denoise” recipe that reduces the burden of learning graph priors from scratch, while retaining the domain-specific inductive bias imposed by the tree and aggregation objectives.

2 Graph on Trees

Definition 1 (Rooted tree). A rooted tree is a tuple $T = (V_T, E_T, r)$ where V_T is a finite set of nodes, $E_T \subseteq \{\{u, v\} : u, v \in V_T, u \neq v\}$ is an undirected edge set such that (V_T, E_T) is a connected acyclic graph, and $r \in V_T$ is the distinguished root.

For a rooted tree $T = (V_T, E_T, r)$, let $\text{par}_T(v)$ denote the parent of $v \neq r$, and define the ancestor relation $\preceq_T$ by $u \preceq_T v$ iff u lies on the unique simple path from r to v (so u is an ancestor of v). Define the depth $\text{dep}_T(v)$ as the length (number of edges) of the path from r to v . For $u \in V_T$, define the (rooted) subtree

$$\text{Sub}_T(u) := \{v \in V_T : u \preceq_T v\},$$

and the set of leaves

$$\text{Leaf}(T) := \{v \in V_T : v \text{ has no children in } T\}.$$

Definition 2 (Graph-on-Trees, GoT). A Graph-on-Trees object is a quadruple

$$\mathcal{G} = (T, V, E, \tau)$$

where:

- $T = (V_T, E_T, r)$ is a rooted tree (the hierarchy);
- $V \subseteq V_T$ is a set of active nodes (allowed to appear as graph nodes), possibly spanning multiple depths;
- $E \subseteq \{\{u, v\} : u, v \in V, u \neq v\}$ is an undirected edge set on V (the relational edges laid over the tree);
- $\tau : E \rightarrow \mathcal{Y}$ is an optional edge annotation function (e.g. weights, types, timestamps), with codomain $\mathcal{Y}$ (if unneeded, omit τ).

The underlying graph of $\mathcal{G}$ is $G = (V, E)$ and the supporting tree is T .

Definition 3 (Graph of Leaves, GoL). A Graph of Leaves is a Graph on Trees $\mathcal{G} = (T, V, E, \tau)$ such that

$$V \subseteq \text{Leaf}(T).$$

If moreover $V = \text{Leaf}(T)$, we call $\mathcal{G}$ a full Graph of Leaves. The underlying graph of a GoL is $G = (V, E)$, whose vertices are (a subset of) the leaves of T .

Remark 1. Every GoL is a special case of a GoT; while the converse does not hold: a general GoT may include internal (non-leaf) nodes of the supporting trees.

Definition 4 (Directed Graph on Trees, D-GoT). A Directed Graph on Trees (D-GoT) object is a tuple

$$\mathcal{G}^\rightarrow = (T, V, E^\rightarrow, \tau),$$

where:

- $T = (V_T, E_T, r)$ is a rooted tree (the supporting hierarchy);

- $V \subseteq V_T$ is a set of active nodes (graph vertices), possibly spanning multiple depths;
- $E^\rightarrow \subseteq V \times V \setminus \{(v, v) : v \in V\}$ is a set of directed edges;
- $\tau : E^\rightarrow \rightarrow \mathcal{Y}$ is an optional edge-attribute function (weights, types, timestamps, etc.); if unneeded, omit τ .

The underlying directed graph is $G^\rightarrow = (V, E^\rightarrow)$ and the supporting tree is T .

Definition 5 (Directed Graph of Leaves, D-GoL). A Directed Graph of Leaves (D-GoL) is a D-GoT $\mathcal{G}^\rightarrow = (T, V, E^\rightarrow, \tau)$ such that

$$V \subseteq \text{Leaf}(T).$$

If moreover $V = \text{Leaf}(T)$, we call it a full D-GoL. Its underlying directed graph is $G^\rightarrow = (V, E^\rightarrow)$ with vertices drawn from the leaves of T .

Remark 2. Similarly, a D-GoL is a special D-GoT.

Definition 6 (Temporal GoT, T-GoT). A Temporal Graph on Trees (T-GoT) is a time-indexed family

$$\mathbb{G} = (T, \{(V_t, E_t^\rightarrow, \tau_t)\}_{t \in \mathcal{T}}),$$

where $\mathcal{T}$ is an ordered time index set (e.g., $\mathbb{Z}$, $\mathbb{R}$, or a set of timestamps), and for each $t \in \mathcal{T}$ the quadruple

$$\mathcal{G}_t^\rightarrow = (T, V_t, E_t^\rightarrow, \tau_t)$$

is a D-GoT on the same supporting tree T . Equivalently, a T-GoT may be represented as an event stream

$$\mathcal{E} = \{(t_k, u_k, v_k, y_k)\}_{k=1}^M,$$

with $t_k \in \mathcal{T}$, $(u_k, v_k) \in V_T \times V_T$, and $y_k \in \mathcal{Y}$, together with a rule for constructing $(V_t, E_t^\rightarrow, \tau_t)$ from $\mathcal{E}$ (e.g., active nodes are endpoints of events in a neighborhood of t , and τ_t aggregates event attributes).

Definition 7 (Temporal GoL, T-GoL). Fix a rooted tree $T = (V_T, E_T, r)$ with leaves $\text{Leaf}(T)$ and a time index set $\mathcal{T}$ (e.g., discrete times $\mathbb{Z}$ or continuous times $\mathbb{R}$). A Temporal Graph of Leaves (T-GoL) is a time-indexed family

$$\mathbb{G} = (T, \{(V_t, E_t^\rightarrow, \tau_t)\}_{t \in \mathcal{T}}),$$

such that for every $t \in \mathcal{T}$ the snapshot

$$\mathcal{G}_t^\rightarrow = (T, V_t, E_t^\rightarrow, \tau_t)$$

is a Directed Graph of Leaves (D-GoL), i.e.,

$$V_t \subseteq \text{Leaf}(T), \quad E_t^\rightarrow \subseteq V_t \times V_t \setminus \{(v, v) : v \in V_t\},$$

and $\tau_t : E_t^\rightarrow \rightarrow \mathcal{Y}$ is an optional edge-attribute map.

Equivalently, a T-GoL may be represented by an event stream

$$\mathcal{E} = \{(t_k, \ell_k^{\text{src}}, \ell_k^{\text{dst}}, y_k)\}_{k=1}^M,$$

with $t_k \in \mathcal{T}$, $\ell_k^{\text{src}}, \ell_k^{\text{dst}} \in \text{Leaf}(T)$, $y_k \in \mathcal{Y}$, together with a rule for constructing each snapshot $(V_t, E_t^\rightarrow, \tau_t)$ from events at (or near) time t (e.g., using exact timestamps, sliding windows, or exponential decay).

Definition 8 (Window system). Let $\mathcal{T}$ be an ordered time domain. A window system is a collection $\mathcal{W}$ of subsets $W \subseteq \mathcal{T}$. Common choices include fixed-width sliding windows $W = [t - \Delta, t)$ or disjoint partitions of $\mathcal{T}$ (e.g., calendar months). For $W \in \mathcal{W}$, we write $\mathcal{E}|_W$ for events with timestamps in W .

Definition 9 (Window aggregator). Fix a window $W \in \mathcal{W}$. A window aggregator is a mapping

$$\text{Agg}_W : \{(t, u, v, y) \text{ multiset}\} \rightarrow (E_W^\rightarrow, \tau_W),$$

which converts the multiset of directed events in W into a directed edge set and edge attributes. For example, if y contains an amount, Agg_W may sum amounts per ordered pair (u, v) and drop pairs with zero total.

Definition 10 (Windowed GoL, W-GoL). Let $\mathbb{G}$ be a T-GoT represented by an event stream $\mathcal{E}$ on a supporting tree T . A Windowed Graph of Leaves (W-GoL) over a window system $\mathcal{W}$ is a family

$$\{\mathcal{G}_W^\rightarrow\}_{W \in \mathcal{W}}, \quad \mathcal{G}_W^\rightarrow = (T, V_W, E_W^\rightarrow, \tau_W),$$

such that:

- $V_W \subseteq \text{Leaf}(T)$ (vertices are leaves);
- $(E_W^\rightarrow, \tau_W) = \text{Agg}_W(\mathcal{E}|_W)$, i.e., edges/attributes are obtained by aggregating events whose timestamps lie in W (restricted to endpoints in V_W when needed).

If $V_W = \text{Leaf}(T)$ for all W , we call it a full W-GoL.

Definition 11 (Windowed GoT, W-GoT). Let $\mathbb{G}$ be a T-GoT represented by an event stream $\mathcal{E}$ on a supporting tree T . A Windowed Graph on Trees (W-GoT) over a window system $\mathcal{W}$ is a family

$$\{\mathcal{H}_W^\rightarrow\}_{W \in \mathcal{W}}, \quad \mathcal{H}_W^\rightarrow = (T, \tilde{V}_W, \tilde{E}_W^\rightarrow, \tilde{\tau}_W),$$

such that:

- $\tilde{V}_W \subseteq V_T$ (vertices may include internal nodes);
- $(\tilde{E}_W^\rightarrow, \tilde{\tau}_W) = \widetilde{\text{Agg}}_W(\mathcal{E}|_W)$ for some aggregator $\widetilde{\text{Agg}}_W$ that may (optionally) include hierarchical coarsening, e.g. mapping event endpoints to ancestors before aggregating.

A common construction is to choose a (possibly window-dependent) coarsening map

$$a_W : \text{Leaf}(T) \rightarrow V_T \quad \text{with} \quad a_W(\ell) \preceq_T \ell,$$

set $\tilde{V}_W = \{a_W(\ell) : \ell \in \text{Leaf}(T)\}$ (or a subset), map each leaf-level event (t, ℓ_i, ℓ_j, y) to $(t, a_W(\ell_i), a_W(\ell_j), y)$, and then apply a standard aggregator on mapped endpoints.

3 Graph-on-Trees Diffusion Model

3.1 Problem Setting

Many accounting datasets are naturally *temporal event streams* (journal entries / postings) rather than dense graphs at each instant. We therefore adopt a *window* construction that aggregates events into a graph snapshot per time window.

Event stream on a chart-of-accounts tree. Let $T = (V_T, E_T, r)$ be a rooted account tree (chart of accounts) and let $L \subseteq \text{Leaf}(T)$ denote the set of posting accounts (leaves). We model raw data as a multiset of directed events

$$\mathcal{E} = \{(t_k, \ell_k^{\text{src}}, \ell_k^{\text{dst}}, y_k)\}_{k=1}^M,$$

where $t_k \in \mathcal{T}$ is a timestamp, $\ell_k^{\text{src}}, \ell_k^{\text{dst}} \in L$ are (directed) endpoints (e.g., debit→credit projection, or another domain-specific direction convention), and $y_k \in \mathcal{Y}$ are event attributes (e.g., amount, currency, journal type).

Window system and aggregation. Let $\mathcal{W}$ be a collection of time windows $W \subseteq \mathcal{T}$ (e.g., disjoint calendar months or sliding windows). For a window $W \in \mathcal{W}$, write $\mathcal{E}|_W = \{(t, \ell^{\text{src}}, \ell^{\text{dst}}, y) \in \mathcal{E} : t \in W\}$. A window aggregator

$$\text{Agg}_W : \mathcal{E}|_W \longrightarrow (E_W, \tau_W)$$

maps the multiset of events in W to a directed edge set $E_W \subseteq L \times L$ and edge attributes $\tau_W : E_W \rightarrow \mathcal{Y}'$ (e.g., summing amounts and counts per ordered pair).

Windowed Graph of Leaves (W-GoL). The graph snapshot for window W is the GoL

$$\mathcal{G}_W = (T, L, E_W, \tau_W), \quad (E_W, \tau_W) = \text{Agg}_W(\mathcal{E}|_W).$$

We refer to $\{\mathcal{G}_W\}_{W \in \mathcal{W}}$ as a *windowed GoL* representation of the event stream.

Training dataset. A training dataset is a collection of windowed GoLs

$$\mathcal{D} = \{\mathcal{G}_W^{(n)}\}_{n=1}^N,$$

where each sample $\mathcal{G}_W^{(n)} = (T^{(n)}, L^{(n)}, E_W^{(n)}, \tau_W^{(n)})$ corresponds to one entity (or ledger) and one time window. For notational simplicity, we write a generic sample as

$$\mathcal{G}_0 = (T, L, E_0, \tau_0),$$

with the understanding that $\mathcal{G}_0$ is a *windowed snapshot*.

3.2 Forward corruption: GoL $\rightarrow$ GoT

GoT-D defines a family of stochastic forward kernels

$$q_t(\mathcal{G}_t \mid \mathcal{G}_0), \quad t = 1, \dots, T,$$

which transform a leaf graph $\mathcal{G}_0$ into a *Graph on Trees* (GoT)

$$\mathcal{G}_t = (T, V_t, E_t, \tau_t), \quad V_t \subseteq V_T,$$

by progressively *coarsening* nodes upward along the tree and *aggregating* edges accordingly. The forward process is designed to be information-destroying (to prevent trivial inversion) while remaining explainable (via aggregation constraints).

Noise schedules and anti-collapse guard. We fix monotone schedules $\{p_t^{\text{lift}}\}$, $\{q_t^{\text{drop}}\}$, and $\{r_t^{\text{add}}\}$ controlling, respectively, node-lifting, edge deletion, and spurious edge addition at timestep t . We also fix a maximum lift distance $m_t \in \mathbb{N}$ (typically $m_t = 1$) and a minimum active-node threshold $K_{\min}$ to prevent collapse to an overly small set of internal nodes.

Leaf-to-ancestor mapping. Let $a_t : L \rightarrow V_T$ be a stochastic mapping from each leaf to an ancestor in T , with $a_0(\ell) = \ell$. Given a_{t-1} , we sample a_t by applying up to m_t parent steps to each leaf’s current image: for each $\ell \in L$, set $v \leftarrow a_{t-1}(\ell)$ and repeat up to m_t times: with probability p_t^{lift} set $v \leftarrow \text{par}_T(v)$ (if $v \neq r$), otherwise keep v . Finally set $a_t(\ell) = v$. The active node set at time t is then

$$V_t := \{a_t(\ell) : \ell \in L\}.$$

If the proposed lifting would reduce $|V_t|$ below $K_{\min}$, we halt further lifting for that step.

Edge aggregation under coarsening. Define the mapped endpoints of an original leaf edge $(\ell_i, \ell_j) \in E_0$ as

$$(u, v) = (a_t(\ell_i), a_t(\ell_j)).$$

Multiple leaf edges may map to the same (u, v) ; these are *collapsed* and their attributes aggregated to produce a directed (or undirected) edge set E_t^{agg} with attributes τ_t^{agg} . Concretely, for each pair (u, v) , let

$$S_{uv}^{(t)} := \{(\ell_i, \ell_j) \in E_0 : a_t(\ell_i) = u, a_t(\ell_j) = v\}.$$

Then $(u, v) \in E_t^{\text{agg}}$ iff $S_{uv}^{(t)} \neq \emptyset$, and $\tau_t^{\text{agg}}(u, v)$ is computed from $\{\tau_0(\ell_i, \ell_j) : (\ell_i, \ell_j) \in S_{uv}^{(t)}\}$ (e.g., sums for counts/weights, histograms for types).

Edge noise. From $(V_t, E_t^{\text{agg}}, \tau_t^{\text{agg}})$ we obtain the final corrupted GoT $\mathcal{G}_t = (T, V_t, E_t, \tau_t)$ by (i) dropping each edge in E_t^{agg} independently with probability q_t^{drop} and (optionally) (ii) adding a small number of spurious edges between random node pairs in $V_t \times V_t$ according to r_t^{add} with a special noise attribute or small random weight.

3.3 Reverse model: denoising GoT $\rightarrow$ GoL

Because the forward map is many-to-one, the reverse model is naturally probabilistic. We learn a denoiser f_θ that, given $(\mathcal{G}_t, t)$, predicts a distribution over leaf-level edges:

$$\hat{p}_{ij} = \Pr_\theta((\ell_i, \ell_j) \in E_0 \mid \mathcal{G}_t, t), \quad \ell_i, \ell_j \in L.$$

In practice, f_θ is implemented by a hierarchy-aware encoder followed by a leaf-edge decoder.

Hierarchy-aware encoder. The encoder performs message passing on a combined relational structure consisting of (i) tree edges (parent–child) and (ii) observed relational edges E_t (possibly typed via τ_t), to produce embeddings for active nodes and leaves. Leaves that are not active at time t obtain representations via downward propagation along the tree and/or ancestor context.

Leaf-edge decoder. Given leaf embeddings $\{z_\ell\}_{\ell \in L}$ and timestep t , the decoder computes a score $s_{ij} = \phi(z_{\ell_i}, z_{\ell_j}, t)$ and sets $\hat{p}_{ij} = \sigma(s_{ij})$. To avoid quadratic cost, training uses negative sampling over leaf pairs.

3.4 Training objectives

At each iteration we sample $\mathcal{G}_0 \sim \mathcal{D}$ and a timestep $t \sim \text{Unif}\{1, \dots, T\}$, then sample $\mathcal{G}_t \sim q_t(\cdot \mid \mathcal{G}_0)$ and predict $\{\hat{p}_{ij}\}$ (and optionally attributes). We optimize a loss that combines local reconstruction on leaves and global explainability at the corrupted resolution.

Leaf-edge reconstruction loss. Let $P = E_0$ be positive leaf edges and let N be a sampled set of negative leaf pairs (non-edges). We minimize the binary cross-entropy

$$\mathcal{L}_{\text{edge}} = - \sum_{(\ell_i, \ell_j) \in P} \log \hat{p}_{ij} - \sum_{(\ell_i, \ell_j) \in N} \log(1 - \hat{p}_{ij}).$$

For weighted edges, $\mathcal{L}_{\text{edge}}$ can be replaced by an appropriate likelihood (e.g., Poisson/NB).

Aggregation (hierarchical consistency) loss. To prevent “leaf hallucinations” that do not explain the observed coarsened GoT, we re-aggregate the predicted leaf edges using the *training-time* mapping a_t (not revealed to the model). Let $\hat{w}_{ij}$ denote a predicted expected leaf weight (e.g., $\hat{w}_{ij} = \hat{p}_{ij}$ or an explicit weight head). For each $(u, v) \in V_t \times V_t$ define the predicted aggregated weight

$$\widehat{W}_{uv} = \sum_{(\ell_i, \ell_j) \in L \times L: a_t(\ell_i)=u, a_t(\ell_j)=v} \hat{w}_{ij}.$$

Let W_{uv} denote the observed aggregated weight derived from $\tau_t(u, v)$ for $(u, v) \in E_t$. We impose

$$\mathcal{L}_{\text{agg}} = \sum_{(u, v) \in E_t} \|\widehat{W}_{uv} - W_{uv}\|_1,$$

with variants using ℓ_2 loss or KL divergence depending on the attribute semantics.

Optional invariants. To encourage the model to use $\mathcal{G}_t$ rather than only a dataset prior, we may penalize deviations in global mass:

$$\mathcal{L}_{\text{mass}} = \left| \sum_{(\ell_i, \ell_j) \in L \times L} \hat{w}_{ij} - \sum_{(\ell_i, \ell_j) \in E_0} w_{ij} \right|.$$

Total objective. The total loss is

$$\mathcal{L} = \mathcal{L}_{\text{edge}} + \lambda_{\text{agg}} \mathcal{L}_{\text{agg}} + \lambda_{\text{mass}} \mathcal{L}_{\text{mass}}.$$

3.5 Training procedure and curriculum

We recommend a curriculum to mitigate optimization instability: (i) begin with a single-step denoiser at large corruption (sample t near T) to ensure the model learns non-trivial hierarchy-aware reconstruction; (ii) then train with t uniformly sampled from $\{1, \dots, T\}$ to obtain robustness across noise levels; (iii) optionally, use the learned denoiser iteratively to sample reconstructions starting from heavily corrupted graphs.

Summary. GoT-D trains a hierarchy-aware denoiser by repeatedly (1) sampling a GoL, (2) corrupting it into a mixed-depth GoT via tree-guided coarsening and edge aggregation, and (3) minimizing a reconstruction loss on leaf edges together with an aggregation-consistency loss that enforces explainability of the corrupted observation.

4 GoT-D Based on a Pretrained Model

While Section 3 presents GoT-D with a generic hierarchy-aware denoiser f_θ , in practice we may initialize (part of) f_θ using a pretrained graph foundation model (GFM) to exploit transferable structural priors. This is particularly appealing when labeled supervision is limited and the training signal is primarily self-supervised reconstruction from corrupted views.

4.1 Pretrained encoder backbone

Let g_ϕ denote a pretrained graph encoder (e.g., a graph foundation model) that maps a graph input to node representations. In our setting, we construct at timestep t an *encoder input graph*

$$H_t = (V_H, E_H, X_t),$$

where V_H contains (at minimum) all leaves L and the corrupted active nodes V_t , and optionally all tree nodes V_T . The edge set combines (i) the supporting tree edges and (ii) the corrupted relational edges:

$$E_H := E_{\text{tree}} \cup E_t, \quad E_{\text{tree}} := \{(u, \text{par}_T(u)), (\text{par}_T(u), u) : u \in V_T, u \neq r\}.$$

Because many pretrained encoders implement neighborhood interaction via an adjacency mask that is effectively undirected, we form a *symmetrized* version of the corrupted relational edges

$$E_t^{\text{sym}} := E_t \cup \{(v, u) : (u, v) \in E_t\},$$

and include a direction indicator (and other transactional attributes) in the edge or node features to preserve asymmetry (see below). The feature matrix X_t concatenates tree-derived structural features and transaction summaries.

Node features. To adapt a pretrained encoder to account graphs, we provide informative node features even when raw identifiers are unavailable. Typical tree-derived features include depth, subtree size, height-to-leaf, and a leaf indicator. If account metadata is available (e.g., account type/category), it is encoded categorically. We also attach transaction statistics computed from the corrupted graph at time t (e.g., in-/out-degree, weighted in-/out-flow), which helps the encoder condition on the corruption state.

4.2 GoT-D denoiser with pretrained encoder

We define the denoiser as a composition of a pretrained encoder and a task-specific decoder:

$$f_\theta(\mathcal{G}_t, t) \equiv h_\psi(g_\phi(H_t), t), \quad \theta := (\phi, \psi),$$

where g_ϕ produces node embeddings $\{z_v\}_{v \in V_H}$ and h_ψ is a lightweight decoder that maps leaf embeddings to a distribution over leaf-level edges. Concretely, for candidate directed leaf pairs $(\ell_i, \ell_j) \in L \times L$ we predict

$$\hat{p}_{ij} = \Pr_\theta((\ell_i, \ell_j) \in E_0 \mid \mathcal{G}_t, t), \quad \hat{w}_{ij} = \mathbb{E}_\theta[w_{ij} \mid \mathcal{G}_t, t],$$

via a directed pairwise decoder

$$h_{ij} = \text{MLP}([z_{\ell_i}; z_{\ell_j}; z_{\ell_i} \odot z_{\ell_j}; e(t)]), \quad \hat{p}_{ij} = \sigma(u^\top h_{ij}), \quad \hat{w}_{ij} = \text{softplus}(v^\top h_{ij}),$$

where $e(t)$ is a learned time embedding, $\odot$ denotes elementwise product, and σ is the logistic sigmoid. Directionality is captured by the ordered pair (ℓ_i, ℓ_j) and may be further reinforced by including a direction flag in the edge-feature construction of H_t .

4.3 Training objectives

The training losses follow Section 3.4: (i) a leaf-edge reconstruction loss $\mathcal{L}_{\text{edge}}$ (with negative sampling) and (ii) an aggregation-consistency loss $\mathcal{L}_{\text{agg}}$ that re-aggregates predicted leaf edges using the training-time mapping a_t to match the observed corrupted edges at time t . Optionally, we include the global mass regularizer $\mathcal{L}_{\text{mass}}$. The total loss remains

$$\mathcal{L} = \mathcal{L}_{\text{edge}} + \lambda_{\text{agg}} \mathcal{L}_{\text{agg}} + \lambda_{\text{mass}} \mathcal{L}_{\text{mass}}.$$

Fine-tuning protocol. A stable strategy is to *warm-start* training by freezing the pretrained encoder parameters ϕ and optimizing only the decoder parameters ψ for several epochs, then unfreezing a subset of top layers of g_ϕ and continuing end-to-end training with a smaller learning rate on ϕ . We retain the curriculum from Section 3.5: begin with larger corruption levels (large t) and expand to uniform sampling of t .

5 Conclusions

Acknowledgments and Disclosure of Funding

References